Documento Técnico do Projeto

Vai e Volta

Locação P2P de Itens Ordinários

Disciplina *Projeto e Desenvolvimento de Software II* — FURG

Campo	Valor
Versão	1.0
Data	10/10/2026
Gestão	Lincon Avila, Miguel Casarin, Pietro Noble e Lucas Derick
Responsáveis pelo documento	**Lincon Avila** (Tech Lead) e **Miguel Casarin** (Product Owner)
Público	Equipe de desenvolvimento
Documento complementar	Kit do Desenvolvedor (processo e fluxo de trabalho)

Sumário

1 Introdução **3**
- 1.1 Público e uso . . . 3
- 1.2 Documentos relacionados . . . 3
- 1.3 Convenções . . . 3

2 Escopo do MVP **3**
- 2.1 O que o MVP contém . . . 3
- 2.2 Natureza do MVP e implicações técnicas . . . 4
- 2.3 Fora do MVP . . . 4

3 Funcionalidades **5**

4 Regras de negócio **7**
- 4.1 Ciclo de vida do aluguel . . . 12

5 Notificações **12**

6 Stack tecnológica **13**

7 Arquitetura **14**
- 7.1 Componentes e fluxo . . . 14
- 7.2 Decisões de arquitetura . . . 14
- 7.3 Tarefas em segundo plano (`worker`) . . . 15
- 7.4 Portas . . . 15

8 Estrutura do repositório e padrões de código **16**
- 8.1 Estrutura de pastas . . . 16
- 8.2 Idioma dos nomes . . . 16
- 8.3 Formatação e lint . . . 16
- 8.4 Organização de módulos . . . 17
- 8.5 Erros e mensagens . . . 17
- 8.6 Logs . . . 17
- 8.7 Dependências . . . 17
- 8.8 Tempo e datas . . . 17

9 Modelo de dados **17**
- 9.1 Diagrama . . . 18
- 9.2 Entidades . . . 18
- 9.3 Regras garantidas pelo banco . . . 20
- 9.4 Evolução do schema . . . 21
- 9.5 Retenção e exclusão de conta . . . 21

10 Contrato da API **21**
- 10.1 Convenções . . . 21
- 10.2 Resumo dos endpoints . . . 21
- 10.3 Detalhes por endpoint (validações na ordem aplicada) . . . 22
- 10.4 Códigos HTTP . . . 25
- 10.5 Catálogo de códigos de erro . . . 25
- 10.6 Fonte do contrato . . . 26

11 Frontend **26**
- 11.1 Mapa de rotas e acesso . . . 26
- 11.2 Estado e acesso à API . . . 26
- 11.3 Componentes reutilizáveis . . . 27
- 11.4 Build e configuração . . . 27
- 11.5 Acessibilidade e responsividade . . . 27

12 Autenticação e segurança **27**

12.1 Autenticação e sessão ... 27
12.2 OWASP Top 10 aplicado ao MVP ... 28
12.3 Dados pessoais (LGPD) ... 28
12.4 Checklist de endurecimento para exposição fora da máquina local ... 29
13 Upload e armazenamento de imagens 29
14 Ambiente e conteinerização 29
14.1 Variáveis de ambiente ... 29
14.2 Serviços ... 30
14.3 Dockerfiles ... 30
14.4 Nginx (`frontend/nginx.conf`) ... 31
14.5 Compose ... 31
14.6 Comandos oficiais ... 31
14.7 Dados de exemplo (seed) ... 32
14.8 Problemas comuns ... 32
15 Testes e validação técnica 33
15.1 Níveis e ferramentas ... 33
15.2 Dados de teste e isolamento ... 33
15.3 Roteiros de ponta a ponta ... 33
15.4 Checklist de verificação do ambiente ... 34
15.5 Cobertura e relatórios ... 34
16 Integração contínua (CI) 34
16.1 Detalhes dos jobs ... 35
16.2 Esqueleto do workflow ... 35
16.3 Custo, permissões e proteções ... 36
16.4 Segredos e rotação ... 36
17 Ambientes e operação 36
17.1 Subir a demonstração ... 37
17.2 Rollback ... 37
17.3 Backup e restauração ... 37
17.4 Logs e monitoramento mínimo ... 37
17.5 Reset de dados ... 37
17.6 Moderação ... 37
18 Limitações conhecidas 37
19 Decisões técnicas adotadas 38
Apêndice A — Usuários de exemplo 40
Histórico de revisões 40

1 Introdução

Este documento é a especificação técnica do **Vai e Volta**, uma plataforma de locação entre pessoas (P2P) de itens ordinários, como ferramentas, eletrônicos e equipamentos. Ele descreve o que o sistema faz (funcionalidades e regras de negócio) e como ele é construído, testado e executado: tecnologias, arquitetura, modelo de dados, contrato da API, segurança, infraestrutura e testes. O sistema será desenvolvido do zero a partir desta especificação.

1.1 Público e uso

O documento é destinado à equipe de desenvolvimento. Cada regra de negócio tem um código (`RN-nn`) e cada funcionalidade tem um código (`F-nn`), usados em todo o texto, nos testes e nas mensagens de commit e de Pull Request quando relevante.

1.2 Documentos relacionados

O **Kit do Desenvolvedor** complementa este documento e descreve o processo de trabalho: branches, commits, Pull Requests, revisão e aceite, Definition of Ready e Definition of Done, estados das tarefas, tratamento de bugs e impedimentos, comunicação e versionamento das releases. Os dois documentos devem ser lidos em conjunto; este define o que construir e como o sistema é estruturado, e o Kit define como o trabalho é conduzido.

1.3 Convenções

- Identificadores técnicos (tabelas, colunas, rotas da API, variáveis, funções) são escritos em inglês; textos exibidos ao usuário ficam em português.
- Em todas as tabelas de regras, a coluna **Onde é garantida** indica **Banco**, **API** ou **UI**. A interface ajuda o usuário, mas nunca é a única garantia de uma regra.
- Datas de aluguel usam o formato `YYYY-MM-DD` na API e `dd/mm/aaaa` na tela; valores monetários são em reais (BRL).

2 Escopo do MVP

O Vai e Volta tem **um único tipo de conta**: a mesma pessoa anuncia itens (papel **anunciante**) e aluga itens de outras pessoas (papel **locatário**). Todo aluguel começa como **proposta** e só vale quando o dono aceita. **Nada é cobrado no MVP.**

Papel	Quem é	Telas principais
Locatário	Quem aluga um item	Feed, página do item, Aluguéis, notificações
Anunciante	Quem coloca um item para alugar	Anunciar item, Meus anúncios, Aluguéis, notificações
Ambos	Qualquer usuário logado	Cadastro, login, perfil e configurações

2.1 O que o MVP contém

Área	Funcionalidades
Conta e acesso	F-01 a F-08

Área	Funcionalidades
Perfil e configurações	F-09 a F-14
Feed e busca	F-15 a F-27
Página do item	F-28 a F-31
Locatário	F-32 a F-37
Anunciante	F-38 a F-41
Tela Aluguéis e anúncios	F-42 a F-45
Notificações	F-46 a F-52

2.2 Natureza do MVP e implicações técnicas

Não há pagamento nem verificação de identidade, mas **CPF, telefone e endereço são dados pessoais (LGPD)**. Por isso o MVP não é simplificado em segurança: senha com hash forte, sessão assinada, controle de visibilidade dos dados (RN-15, RN-17, RN-35), exclusão com retenção de 1 ano (RN-21) e logs sem dados pessoais.

Três pontos têm impacto técnico direto: (a) existem rotinas em segundo plano (expirar propostas em 48 h, avisos de calendário e anonimização após 1 ano); (b) "hoje" e "atrasado" dependem de data e fuso (America/Sao_Paulo, RN-69); (c) o conflito de datas é garantido **pelo banco**, não só pela API (RN-42).

2.3 Fora do MVP

Os itens abaixo **não** são entregues no MVP e não devem ser implementados nem assumidos como existentes.

Funcionalidades

Item	Observação
Distância e filtro por região ("Perto de você")	O local de retirada é exibido, mas não há cálculo de distância
Dono cancelar ou retomar o item durante uma locação	O aluguel aceito só termina com a devolução ou no fim do período (RN-51)
Recuperação de senha	Quem esquece a senha perde o acesso à conta
Termos de uso, idade mínima de 18 anos e lista de itens proibidos	O cadastro não coleta aceite de termos nem data de nascimento
Cobrança, caução, seguro e multa por atraso ou dano	O atraso aparece apenas como situação
Avaliações nos dois sentidos	
Denúncia de anúncio ou usuário e moderação	Remoção de conteúdo indevido é feita manualmente pelo responsável técnico, direto no banco
Prorrogar aluguel	
Limite de aluguéis ativos por pessoa	
Várias unidades no mesmo anúncio	Cada anúncio é um item único
Duplicar anúncio	
Bloquear datas manualmente	
Registro da condição do item na retirada e na devolução; estado de conservação	

Item	Observação
Consequências do atraso (bloqueio de novos aluguéis)	
Regras do dono no anúncio; aviso de fim de janela com renovação	
Contato com o suporte	

Segurança e dados

Item	Motivo
E-mail, telefone e endereço não verificados	Sem integração externa; só o formato é validado
CPF validado só pelos dígitos, sem conferir o titular	Sem integração externa
Sem confirmação por e-mail ao editar dados ou trocar a senha	Exige apenas a senha atual
Sem autenticação em dois fatores e sem revogação individual de sessões	Complexidade fora do escopo
Imagens sem antivírus	Mitigado por validação do conteúdo e reprocessamento (seção 13)

Infraestrutura e operação

- Servidor de deploy e ambiente de teste separado: o sistema roda localmente, via Docker.
- Deploy automático, registro de imagens Docker e publicação de pacotes: não há; as imagens são construídas localmente.
- Backup agendado e monitoramento externo: o backup é manual (seção 17).
- CDN e armazenamento de objetos: as fotos ficam em volume local.
- Exposição fora da máquina local: só com o checklist de endurecimento da seção 12 cumprido.

3 Funcionalidades

Lista das funcionalidades do MVP, agrupadas por área. A numeração é única e usada em todo o documento.

Conta e acesso

Código	Funcionalidade
F-01	Cadastro com nome completo, nome de usuário, e-mail, senha, CPF, telefone e endereço completo
F-02	Aviso de qual campo (e-mail, CPF ou nome de usuário) já está em uso
F-03	Após criar a conta, a pessoa é levada à tela de login
F-04	Validação dos campos na própria tela (CPF, e-mail, usuário, senha, telefone e CEP)
F-05	Entrar com e-mail ou nome de usuário e senha
F-06	Mensagem de erro quando as credenciais não conferem
F-07	Permanecer logado por 15 dias corridos desde o login
F-08	Sair da conta

Perfil e configurações

Código	Funcionalidade
F-09	Ícone de perfil com menu suspenso: Perfil, Configurações e Sair
F-10	Perfil público: nome de usuário, bairro e cidade, membro desde e anúncios ativos
F-11	Configurações: editar dados cadastrais
F-12	Configurações: trocar a senha
F-13	Configurações: excluir a conta
F-14	Reativar a conta excluída ao entrar novamente dentro de 1 ano

Feed e busca

Código	Funcionalidade
F-15	Feed com os itens carregados por rolagem
F-16	Busca pelo nome do item, em qualquer parte do nome
F-17	Filtro por tags: as mais usadas em destaque e um campo para buscar outras
F-18	Faixa de preço com mínimo e máximo digitados
F-19	Atalhos de faixa de preço calculados a partir dos anúncios ativos
F-20	Ordenação por mais recentes, mais antigos, menor preço ou maior preço
F-21	Filtro de disponibilidade: todos, disponíveis agora ou atualmente alugados
F-22	Filtro por período desejado (preciso de ... até ...)
F-23	Card do item: foto, situação, categoria, nome, valor por dia, bairro e tempo desde a publicação
F-24	Contador de itens encontrados
F-25	Categorias fixas em lista lateral
F-26	Favoritar pelo coração no card e filtrar por Favoritos
F-27	Feed sem resultados: mensagem clara, botão para limpar filtros e atalho para anunciar

Página do item

Código	Funcionalidade
F-28	Ver fotos, nome, descrição, valor por dia, prazo máximo, janela, categoria, tags, local de retirada e nome do anunciante
F-29	Ver os períodos já reservados
F-30	Botão para propor o aluguel, que abre a escolha de período
F-31	Visão do dono: situação atual do anúncio e ações de editar e pausar, sem botão de proposta

Locatário

Código	Funcionalidade
F-32	Escolher data de início e de fim do aluguel
F-33	Ver o resumo N dia(s) · R$ X, informativo e sem cobrança
F-34	Enviar a proposta de aluguel ao dono
F-35	Acompanhar a proposta (aguardando, aceita, recusada ou expirada)
F-36	Cancelar proposta ou aluguel

Código	Funcionalidade
F-37	Avisar a devolução antes do prazo previsto

Anunciante

Código	Funcionalidade
F-38	Criar anúncio com nome, descrição, valor, máximo de dias, janela, categoria, tags, fotos e local de retirada
F-39	Receber propostas e aceitar ou recusar
F-40	Marcar um aluguel como devolvido (concluído), inclusive confirmando a devolução antecipada
F-41	Editar o anúncio (todos os campos), pausar, reativar ou excluir, pela tela Meus anúncios

Tela Aluguéis e anúncios

Código	Funcionalidade
F-42	Alternar entre Meus anúncios e Itens que estou alugando
F-43	Em cada visão, ativos (propostas e aluguéis em curso) e histórico
F-44	Em Meus anúncios, ver as propostas recebidas e decidir
F-45	Cada aluguel mostra a situação atual

Notificações

Código	Funcionalidade
F-46	Ícone de sino com indicador de avisos novos
F-47	Caixa de notificações com a lista de avisos
F-48	O dono é avisado quando recebe uma proposta
F-49	O locatário é avisado quando a proposta é aceita ou recusada
F-50	Avisos de aluguel começando, devolução próxima, atraso e proposta expirada
F-51	O dono é avisado quando o locatário comunica devolução antecipada; o locatário, quando o dono conclui o aluguel
F-52	Marcar avisos como lidos, um a um ou todos

4 Regras de negócio

Cada regra indica **onde é garantida** (Banco, API ou UI) e o **teste** reservado para ela (seção 15). Toda regra que depende de "hoje" usa `today_br()` (RN-69).

Cadastro

Código	Regra	Onde é garantida	Teste
RN-01	E-mail, CPF e nome de usuário são únicos. E-mail e usuário não diferenciam maiúsculas de minúsculas. O telefone pode se repetir.	Banco (`UNIQUE`, `citext`) · API (409)	DB-REG-01
RN-02	O e-mail precisa ter formato válido, com até 254 caracteres, e não é verificado no MVP.	API · UI	UT-REG-01

Código	Regra	Onde é garantida	Teste
RN-03	O CPF é guardado com 11 dígitos (só números) e precisa ter dígitos verificadores válidos. O MVP não confirma que pertence à pessoa.	API · UI	UT-REG-02
RN-04	A senha tem de 8 a 128 caracteres, com ao menos uma letra maiúscula, um número e um caractere especial.	API · UI	UT-REG-03
RN-05	O nome de usuário tem de 3 a 30 caracteres: letras minúsculas, números e underscore, sem começar por número nem por underscore.	Banco (`CHECK`) · API · UI	DB-REG-02
RN-06	O nome completo é obrigatório, com até 120 caracteres.	Banco (`NOT NULL`) · API	API-REG-01
RN-07	O telefone tem 11 dígitos (DDD com dois dígitos e celular com nove), gravado só com dígitos. A tela aceita máscara e a API normaliza.	API · UI	UT-REG-04
RN-08	O endereço é obrigatório: CEP (8 dígitos), rua (até 100), número (até 10), complemento (opcional, até 60), bairro (até 100), cidade (até 100) e UF (2 letras, sigla válida). Só o formato do CEP é conferido.	Banco · API	API-REG-02
RN-09	Ao usar e-mail, CPF ou usuário já cadastrado, a resposta indica qual campo está em uso.	API (`fields`) · UI	API-REG-03
RN-10	A senha nunca é exibida nem recuperável por ninguém. Ela é guardada apenas como hash Argon2id.	API	API-REG-04

Login e sessão

Código	Regra	Onde é garantida	Teste
RN-11	O login aceita e-mail ou nome de usuário com a senha; a comparação ignora maiúsculas.	API	API-AUT-01
RN-12	Credenciais inválidas ou conta inexistente geram a mesma mensagem (E-mail, usuário ou senha incorretos), com o mesmo tempo de resposta.	API	API-AUT-02
RN-13	A sessão dura 15 dias corridos desde o login; depois a pessoa é deslogada no navegador e entra de novo.	API · UI	API-AUT-03
RN-14	Após 5 falhas em 15 minutos para o mesmo identificador, o login fica bloqueado por 15 minutos (HTTP 429 com `Retry-After`).	API · Banco (`login_attempts`)	API-AUT-04

Conta e privacidade

Código	Regra	Onde é garantida	Teste
RN-15	E-mail, CPF, telefone, nome completo e endereço completo nunca aparecem no feed, na página do item nem no perfil público. O perfil mostra bairro e cidade.	API (schemas públicos e privados)	API-USR-01
RN-16	Não há confirmação por e-mail ao editar dados ou trocar a senha. Trocar a senha e excluir a conta exigem a senha atual.	API	API-USR-02
RN-17	O telefone só é mostrado à outra parte de um aluguel aceito (`accepted`).	API	API-USR-03
RN-18	A conta não pode ser excluída enquanto houver aluguel aceito, em curso ou futuro, como locatário ou como anunciante (409).	API	API-USR-04
RN-19	Ao excluir a conta, as propostas pendentes são canceladas e a outra parte passa a ver usuário excluído no histórico.	API	API-USR-05

Código	Regra	Onde é garantida	Teste
RN-20	Enquanto a conta está excluída, seus anúncios ficam pausados e ocultos do feed (lista de bloqueio).	API · Banco	API-USR-06
RN-21	Os dados da conta excluída são mantidos por 1 ano. Depois, o worker os anonimiza: usuário `deleted-<id>`, e-mail `anon-<id>@invalid` e demais dados pessoais substituídos por valores inertes.	Worker	API-USR-07
RN-22	Entrar novamente dentro de 1 ano reativa a conta. Os anúncios voltam pausados e o dono os reativa.	API	API-USR-08
RN-23	O histórico de aluguéis fica visível enquanto a conta existir; não há prazo de expiração.	API	API-USR-09

Feed, busca e filtros

Código	Regra	Onde é garantida	Teste
RN-24	O feed mostra anúncios ativos cuja janela ainda não terminou, de donos não excluídos. Os anúncios do próprio usuário aparecem marcados como Seu anúncio.	API	API-FEED-01
RN-25	A busca por nome ignora maiúsculas e acentos, acha o texto em qualquer parte do nome, trata sinais digitados como texto comum e sempre restringe o resultado.	API	API-FEED-02
RN-26	Filtros de tipos diferentes (categoria, tag, preço, disponibilidade, período, favoritos) se combinam com E. Dentro de um mesmo tipo vale OU: o item precisa ter qualquer uma das categorias ou tags escolhidas.	API	API-FEED-03
RN-27	Os atalhos de preço são 3 faixas por tercis dos preços dos anúncios ativos (mais baratos, intermediários e mais caros), recalculadas a cada consulta. Com menos de 3 anúncios ativos, não há atalhos.	API	API-FEED-04
RN-28	Disponíveis agora: anúncio ativo, janela vigente hoje e sem aluguel aceito cobrindo hoje. Atualmente alugados: aluguel aceito em andamento hoje. Todos: os dois grupos.	API	API-FEED-05
RN-29	O filtro de período só vale com início e fim informados e retorna itens cuja janela cobre o período inteiro e sem aluguel aceito sobreposto.	API	API-FEED-06
RN-30	Ao rolar, os itens seguem a ordem escolhida sem repetir nem pular nenhum.	API (cursor por chave)	API-FEED-07
RN-31	Favorito de anúncio pausado ou excluído é mantido, mas fica oculto da lista enquanto o anúncio estiver indisponível.	API	API-FEED-08

Página do item

Código	Regra	Onde é garantida	Teste
RN-32	O valor é por dia, de no mínimo R$ 1,00 e no máximo R$ 5.000,00.	Banco (`CHECK`) · API	DB-ITM-01
RN-33	O dono não vê o botão de proposta no próprio anúncio.	API (`is_owner`) · UI	UT-ITM-01
RN-34	Anúncio pausado, excluído ou de conta excluída deixa de aparecer no feed e sua página mostra anúncio indisponível (exceto para o dono).	API · UI	API-ITM-01
RN-35	A página mostra rua, bairro e cidade do local de retirada; número e complemento só aparecem ao locatário com aluguel aceito.	API	API-ITM-02

Propostas e aluguéis

Código	Regra	Onde é garantida	Teste
RN-36	O período vai da data de início à data de fim, ambas inclusivas: duração = fim − início + 1 dia. Aluguel de 1 dia tem início igual ao fim.	API · Banco (`daterange`)	UT-RNT-01
RN-37	A duração fica entre 1 dia e o prazo máximo do anúncio.	API · UI	API-RNT-01
RN-38	O período deve estar inteiro dentro da janela de disponibilidade do anúncio.	API · UI	API-RNT-02
RN-39	Não se propõe aluguel para datas passadas (início menor que hoje).	API · UI	API-RNT-03
RN-40	Não é possível alugar o próprio item (403).	API	API-RNT-04
RN-41	Todo aluguel começa como proposta e só vale depois que o dono aceita.	API	API-RNT-05
RN-42	Dois aluguéis aceitos do mesmo item não se sobrepõem, com intervalos inclusivos: devolução em 14/10 libera nova retirada só em 15/10.	Banco (`EXCLUDE USING gist`) · API	DB-RNT-01
RN-43	Valor estimado = valor por dia × duração, fixo desde o envio da proposta. É informativo e não gera cobrança.	API · Banco	API-RNT-06
RN-44	Proposta pendente não bloqueia datas: vários locatários podem propor para o mesmo período.	API	API-RNT-07
RN-45	Um locatário não pode ter duas propostas pendentes sobrepostas para o mesmo item (409).	API	API-RNT-08
RN-46	Ao aceitar uma proposta, as outras pendentes que se sobrepõem são recusadas automaticamente e os locatários avisados.	API (mesma transação)	API-RNT-09
RN-47	A proposta expira se o dono não responder em 48 horas, ou se a data de início chegar sem resposta. O locatário é avisado.	Worker · API	API-RNT-10
RN-48	O aceite exige início maior ou igual a hoje; caso contrário a resposta é 409 e a proposta passa a `expired`.	API	API-RNT-11
RN-49	O locatário cancela a proposta pendente a qualquer momento e o aluguel aceito até a véspera do início.	API	API-RNT-12
RN-50	O locatário pode devolver antes do fim: ele avisa, o dono confirma, o período restante é liberado e o valor informativo é recalculado pelos dias usados.	API	API-RNT-13
RN-51	O dono não cancela nem revoga aluguel aceito: ele termina com a devolução ou no fim do período. Se precisar do item, combina com o locatário pelo telefone.	API (sem endpoint)	API-RNT-14
RN-52	Locatário com aluguel atrasado pode propor outros aluguéis; o atraso não gera multa nem bloqueio.	API	API-RNT-15

Anúncios

Código	Regra	Onde é garantida	Teste
RN-53	Obrigatórios: nome (até 80), descrição (até 1.000), valor, máximo de dias (1 ou mais), janela com início menor ou igual ao fim, categoria e ao menos 1 foto.	API · UI	API-ADS-01
RN-54	O fim da janela não pode estar no passado.	API	API-ADS-02
RN-55	O máximo de dias não pode ser maior que o tamanho da janela; o sistema recusa o anúncio em vez de ajustar sozinho.	Banco (`CHECK`) · API	DB-ADS-01
RN-56	Fotos: de 1 a 5 por anúncio, até 5 MB cada, JPEG, PNG ou WebP. A primeira é a capa.	API	API-ADS-03

Código	Regra	Onde é garantida	Teste
RN-57	O anunciante cria até 5 tags por anúncio, com até 30 caracteres. Tags que diferem só em maiúsculas ou acentos são a mesma.	API · Banco	API-ADS-04
RN-58	Cada anúncio tem uma categoria fixa obrigatória: Ferramentas, Eletrônicos, Esportes e lazer, Casa e jardim, Equipamentos, Veículos, Livros e Festas e jogos.	Banco (FK) · API	DB-ADS-02
RN-59	O local de retirada é, por padrão, o endereço do cadastro e pode ser editado em cada anúncio.	API	API-ADS-05
RN-60	Só o dono edita, pausa, reativa ou exclui o anúncio (403).	API	API-ADS-06
RN-61	Todos os campos do anúncio podem ser editados, e a edição vale só para novos locatários. Cada proposta guarda uma cópia do anúncio no envio; aluguéis em curso mantêm essa versão até terminar.	API · Banco (`listing_snapshot`)	API-ADS-07
RN-62	Anúncio com aluguel aceito em curso ou futuro não pode ser pausado, excluído nem ter a janela encurtada (409). Ao pausar ou excluir, as propostas pendentes são recusadas.	API	API-ADS-08
RN-63	Só o dono aceita ou recusa uma proposta, e a recusa não exige motivo.	API	API-ADS-09
RN-64	Só o dono marca como devolvido. Concluído, cancelado, recusado e expirado liberam o período.	API	API-ADS-10

Situação e histórico

Código	Regra	Onde é garantida	Teste
RN-65	A situação é calculada pelas datas: aguardando, reservado, em andamento, atrasado, concluído, cancelado, recusado e expirado (tabela na seção 4.1).	API (função única)	UT-LST-01
RN-66	O histórico reúne aluguéis concluídos, cancelados, recusados e expirados; o restante é ativo.	API	API-LST-01

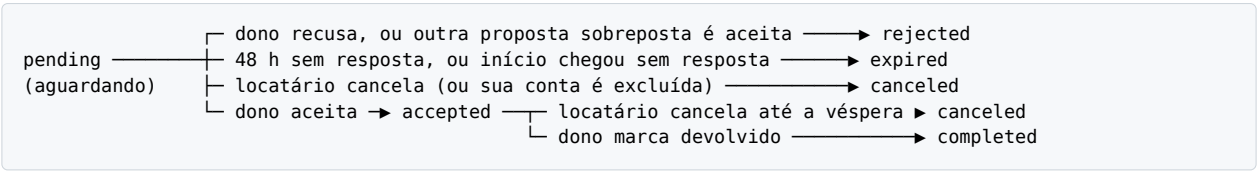
Notificações

Código	Regra	Onde é garantida	Teste
RN-67	Cada aviso leva à tela relacionada (por exemplo, proposta recebida abre a tela Aluguéis).	UI	UT-NTF-01
RN-68	Os avisos têm estado lido e não lido, e o sino indica se há novos.	API · UI	API-NTF-01

Regras gerais

Código	Regra	Onde é garantida	Teste
RN-69	Moeda sempre real (BRL) e fuso America/Sao_Paulo; todo hoje e atrasado usam `today_br()`.	API · UI	UT-GER-01

4.1 Ciclo de vida do aluguel



A **situação exibida** (RN-65) é uma função única de (`status`, `start_date`, `end_date`, hoje), implementada em um só lugar do backend:

Situação	Condição
Aguardando	`pending`, com `expires_at` no futuro e `start_date` maior ou igual a hoje
Reservado	`accepted` e `start_date` > hoje
Em andamento	`accepted` e `start_date` ≤ hoje ≤ `end_date`
Atrasado	`accepted` e `end_date` < hoje
Concluído · Cancelado · Recusado · Expirado	`completed` · `canceled` · `rejected` · `expired` (inclui `pending` vencida)

Histórico (RN-66): `completed`, `canceled`, `rejected` e `expired`. Todo o resto é ativo. Só o aluguel `accepted` bloqueia datas; reservado, em andamento e atrasado são situações calculadas pelas datas.

- **Aluguel de 1 dia:** `start_date = end_date`; retirada e devolução no mesmo dia. A devolução no último dia só libera nova retirada no dia seguinte (RN-42).
- **Devolução antecipada** (RN-50, F-37): o locatário chama `return-request`, o que grava `return_requested_at` e avisa o dono. O dono confirma em `complete`: o sistema grava `returned_on`, **encurta `end_date` para a data da devolução**, guarda `original_end_date` e recalcula `estimated_total` pelos dias usados. O período restante fica livre porque o `end_date` passou a ser menor.
- **Conclusão:** só é permitida com `start_date` ≤ hoje. O atraso não gera multa nem bloqueio.
- **Conta excluída** (RN-19): propostas pendentes viram `canceled`; a outra parte vê "usuário excluído" enquanto a conta estiver excluída.
- **Aceite com início passado** (RN-48): se `start_date` < hoje, a API responde 409 e marca a proposta como `expired`.

5 Notificações

Cada aviso é uma linha em `notifications` com `type`, `payload` (JSON) e `read_at` (RN-68). O texto exibido é montado no frontend a partir do tipo e do payload; o banco guarda apenas dados, nunca a frase. O payload não contém e-mail, CPF, telefone, endereço nem nome completo.

Tipo (`type`)	Destinatário	Quando	Payload	Destino do clique
`proposal_received`	Dono	Locatário envia proposta	`rental_id`, `listing_id`, `listing_title`, `renter_username`, `start_date`, `end_date`	`/alugueis` (Meus anúncios)
`proposal_accepted`	Locatário	Dono aceita a proposta	`rental_id`, `listing_id`, `listing_title`, `start_date`, `end_date`	`/alugueis` (Itens que estou alugando)

Tipo (`type`)	Destinatário	Quando	Payload	Destino do clique
`proposal_rejected`	Locatário	Dono recusa, outra proposta sobreposta é aceita ou o anúncio é pausado ou excluído	`rental_id`, `listing_id`, `listing_title`, `reason` (`owner`, `overlap` ou `listing_unavailable`)	`/alugueis` (Itens que estou alugando)
`proposal_expired`	Locatário	Proposta passa de 48 h ou o início chega sem resposta	`rental_id`, `listing_id`, `listing_title`	`/alugueis` (Itens que estou alugando)
`rental_starting`	Ambos	Véspera do início (`start_date` = amanhã)	`rental_id`, `listing_id`, `listing_title`, `start_date`, `role` (`renter` ou `owner`)	`/alugueis`
`return_due`	Ambos	Véspera do fim (`end_date` = amanhã)	`rental_id`, `listing_id`, `listing_title`, `end_date`, `role`	`/alugueis`
`rental_overdue`	Ambos	Dia seguinte ao fim, sem conclusão	`rental_id`, `listing_id`, `listing_title`, `end_date`, `role`	`/alugueis`
`return_requested`	Dono	Locatário avisa devolução antecipada	`rental_id`, `listing_id`, `listing_title`, `renter_username`	`/alugueis` (Meus anúncios)
`rental_completed`	Locatário	Dono marca como devolvido	`rental_id`, `listing_id`, `listing_title`, `returned_on`, `estimated_total`	`/alugueis` (histórico)
`rental_canceled`	Dono ou locatário	Locatário cancela o aluguel aceito, ou a conta da outra parte é excluída	`rental_id`, `listing_id`, `listing_title`, `canceled_by` (`renter` ou `account_deleted`)	`/alugueis` (histórico)

Os tipos `rental_starting`, `return_due` e `rental_overdue` são gerados pelo `worker` e protegidos contra duplicidade por índice único (seção 9). Os demais são gerados pela API na mesma transação da ação que os origina.

6 Stack tecnológica

As versões abaixo são as séries estáveis atuais, pesquisadas em 10/10/2026. O patch exato é resolvido e travado nos arquivos de lock (`uv.lock` e `package-lock.json`); mudanças de série maior só por Pull Request dedicado. **Todas as ferramentas são gratuitas e de código aberto; o projeto não depende de nenhum serviço pago.**

Camada	Tecnologia	Versão	Motivo
Linguagem (back)	Python	3.14	Série estável atual, com bom suporte às bibliotecas escolhidas
Backend	FastAPI + Uvicorn	FastAPI 0.136	Validação por Pydantic e OpenAPI automático (seção 10)
Validação	Pydantic	2.12	Schemas de entrada e saída; modelos públicos e privados separados
ORM e driver	SQLAlchemy + psycopg 3 (`psycopg[binary]`)	2.0 · 3	Suporta constraints de exclusão e SQL puro quando necessário
Migrações	Alembic	1	Evolução do schema (seção 9.4)
Banco	PostgreSQL	18	Exige `EXCLUDE USING gist` (RN-42), `citext`, `pg_trgm`, `unaccent` e `btree_gist`
Hash de senha	Argon2id (`argon2-cffi`)	25.1	Padrão atual recomendado pelo OWASP
Sessão	JWT assinado (`PyJWT`)	2	Sessão de 15 dias sem tabela de sessões (RN-13)
Imagens	Pillow	12	Valida e reprocessa os arquivos (seção 13)

Camada	Tecnologia	Versão	Motivo
Tarefas agendadas	APScheduler em processo `worker` próprio	3.11	Expirar propostas, avisos e anonimização (seção 7)
Dependências (back)	uv (`pyproject.toml` e `uv.lock`)	versão fixada no Dockerfile e no CI	Instalação reprodutível e rápida, com hashes no lock
Runtime (front)	Node.js (LTS ativa)	24	Build do front e ferramentas
Frontend	React + Vite + React Router	19.2 · 8 · 7	SPA com rotas públicas e privadas; JavaScript (JSX)
Dados no front	TanStack Query	5	Cache, rolagem infinita (RN-30) e invalidação
Estilização	Tailwind CSS (plugin `@tailwindcss/vite`)	4	Rapidez e consistência visual
Servidor web	Nginx (imagem alpine)	1.30	Serve a SPA, faz proxy da API e entrega `/uploads`
Contêineres	Docker + Docker Compose v2	atuais	Subir tudo com um comando
Testes (back)	pytest + httpx + freezegun	pytest 9	Integração com PostgreSQL real; relógio congelado
Testes (front)	Vitest + Testing Library; Playwright (E2E)	atuais	Componentes e fluxos principais
Lint e formatação	Ruff (back) · ESLint + Prettier (front)	Ruff 0.15	Uma ferramenta por camada, comando único
Integração contínua	GitHub Actions	—	Gratuito em repositório público e com cota gratuita em privado (seção 16)
Varredura de segredos	Gitleaks (binário de código aberto)	versão fixada	Executado direto no CI, sem a action oficial, que exige licença em organizações

7 Arquitetura

7.1 Componentes e fluxo

```
Navegador ──HTTP──▶ Nginx (web) ─┬─ /          → arquivos estáticos da SPA (fallback para index.html)
                                 ├─ /api/*     → proxy → backend (FastAPI, porta 8000, rede interna)
                                 └─ /uploads/* → arquivos do volume `uploads` (somente leitura)

backend ──SQL──▶ PostgreSQL (db, porta 5432, rede interna)
worker  ──SQL──▶ PostgreSQL     (mesma imagem do backend, outro comando)
backend/worker ──▶ volume `uploads` (backend grava; Nginx lê)
```

Fluxo de uma requisição: o navegador chama `/api/...` na **mesma origem** do site; o Nginx encaminha ao backend; o backend valida o cookie de sessão, aplica as regras, consulta o banco e responde JSON.

7.2 Decisões de arquitetura

Decisão	Motivo
SPA + API REST JSON	Separa interface e regras; contrato único consumido pelo front
Nginx como única porta pública	Mesma origem elimina CORS e esconde backend e banco
Integridade crítica no banco (único, CHECK, EXCLUDE)	Reservas simultâneas não podem depender só de código (RN-42)

Decisão	Motivo
Situação do aluguel **calculada** pelas datas (RN-65)	Evita estados que "esquecem" de virar; função única no backend
`worker` separado do `backend`	Tarefas de tempo (48 h, atraso, 1 ano) rodam sem depender de requisição
Tarefas idempotentes	Reexecutar após falha não duplica aviso nem corrompe dado
Datas de aluguel como `DATE`; instantes como `timestamptz` (UTC)	"Hoje" é calculado em America/Sao_Paulo, sem erro de fuso
Cópia do anúncio em cada proposta (`listing_snapshot`)	Edições do anúncio não alteram aluguéis em curso (RN-61)
Imagens em volume, não no banco	Simples para o MVP; backup separado (seção 17)
Um único ponto de serialização por visibilidade	Telefone e endereço só saem por funções que aplicam RN-15, RN-17 e RN-35

7.3 Tarefas em segundo plano (`worker`)

Tarefa	Regra	Frequência	O que faz
Expirar propostas	RN-47	a cada minuto	`pending` com `expires_at` < agora, ou com `start_date` < hoje, vira `expired` e avisa o locatário
Avisos de calendário	F-50	a cada 15 min	Início (véspera), devolução próxima (véspera do fim) e atraso (fim passou); duplicidade evitada por índice único
Anonimizar contas	RN-21	diária	Contas com `purge_after` < agora têm os dados pessoais substituídos por valores inertes
Limpeza	RN-14, seção 13	diária	Remove tentativas de login com mais de 1 dia e arquivos de foto sem referência (considerando também as cópias de anúncio em aluguéis ativos)

- **Instância única:** a cada ciclo o `worker` obtém um lock consultivo do PostgreSQL (`pg_try_advisory_lock`); se não conseguir, pula o ciclo. Assim, duas instâncias nunca executam a mesma tarefa ao mesmo tempo.
- **Heartbeat:** ao final de cada ciclo o `worker` atualiza o arquivo `/tmp/worker_heartbeat`. O healthcheck do contêiner falha se o arquivo tiver mais de 120 segundos.
- **Independência do worker:** as consultas da API também tratam proposta vencida como expirada, para que a regra valha mesmo com o `worker` parado.

7.4 Portas

Ambiente	Exposto no host
Local (desenvolvimento)	8080 (Nginx), 8000 (API) e 5432 (banco)
Demonstração	Somente o Nginx (`WEB_PORT`, padrão 8080)

8 Estrutura do repositório e padrões de código

8.1 Estrutura de pastas

```
vai-e-volta/
├─ backend/
│  ├─ app/
│  │  ├─ main.py                # criação do app, rotas, handlers de erro
│  │  ├─ config.py              # leitura de variáveis de ambiente
│  │  ├─ routers/               # auth, users, listings, rentals, favorites, notifications, meta
│  │  ├─ schemas/               # modelos Pydantic (entrada e saída; públicos e privados)
│  │  ├─ models/                # tabelas SQLAlchemy
│  │  ├─ services/              # regras de negócio (aluguel, anúncio, conta, situação)
│  │  ├─ core/                  # segurança, sessão, relógio (today_br), paginação por cursor
│  │  ├─ worker/                # tarefas agendadas
│  │  └─ scripts/               # seed_dev.py, export_openapi.py
│  ├─ migrations/               # Alembic (inclui extensões e categorias fixas)
│  ├─ tests/                    # unit/ · integration/ · conftest.py
│  ├─ Dockerfile · .dockerignore
│  └─ pyproject.toml · uv.lock  # dependências, Ruff, pytest
├─ frontend/
│  ├─ src/ (pages/ · components/ · services/ · context/ · hooks/)
│  ├─ tests/                    # unitários (Vitest)
│  ├─ Dockerfile · nginx.conf · .dockerignore
│  └─ package.json · package-lock.json
├─ e2e/                         # Playwright (fluxos de ponta a ponta)
├─ scripts/                     # backup.sh · restore.sh
├─ docs/                        # openapi.json (exportado), diagramas
├─ .github/workflows/ci.yml     # integração contínua
├─ docker-compose.yml           # base (db, backend, worker, web)
├─ docker-compose.override.yml  # desenvolvimento (portas, hot reload)
├─ docker-compose.prod.yml      # demonstração
├─ .env.example
├─ Makefile                     # comandos oficiais (seção 14)
└─ README.md                    # como executar e referência a este documento
```

8.2 Idioma dos nomes

- **Identificadores técnicos em inglês:** tabelas, colunas, arquivos, rotas da API, variáveis e funções (`listings`, `daily_price`, `/api/listings`).
- **Português apenas no que o usuário vê:** textos de tela, mensagens de erro, rótulos de situação e URLs das telas do front (`/alugueis`).
- Códigos de erro em inglês `SNAKE_CASE`; mensagens em português.

8.3 Formatação e lint

Camada	Ferramenta	Comando único (local e CI)
Backend	Ruff (lint e format)	`make lint-backend`
Frontend	ESLint e Prettier	`make lint-frontend`
Ambos	—	`make lint`

A configuração é versionada (`pyproject.toml`, `eslint.config.js`, `.prettierrc`). O CI executa exatamente os mesmos comandos.

8.4 Organização de módulos

- **Backend:** `routers` só recebem e respondem; **regras ficam em `services`**; `schemas` separam modelos público e privado (RN-15); `models` só descrevem tabelas. Nenhum router consulta o banco diretamente para aplicar regra de negócio.
- **Frontend:** `pages` (uma por rota), `components` (reutilizáveis), `services` (único acesso à API), `context` (sessão) e `hooks` (consultas e rolagem).

8.5 Erros e mensagens

Formato único (seção 10.1): `{"error": {"code", "message", "fields"}}`. Erros de validação trazem `fields` com a mensagem por campo (RN-09). O login nunca revela se a conta existe (RN-12). O front exibe `message`, destaca `fields` e nunca mostra detalhes técnicos.

8.6 Logs

- Saída padrão em JSON, com `request_id`, método, rota, status e duração.
- Níveis: `INFO` (requisições e tarefas do `worker`), `WARNING` (login bloqueado, conflito de reserva) e `ERROR` (exceções).
- **Nunca registrar** senha, CPF, e-mail, telefone, endereço, token ou corpo de requisição (LGPD).

8.7 Dependências

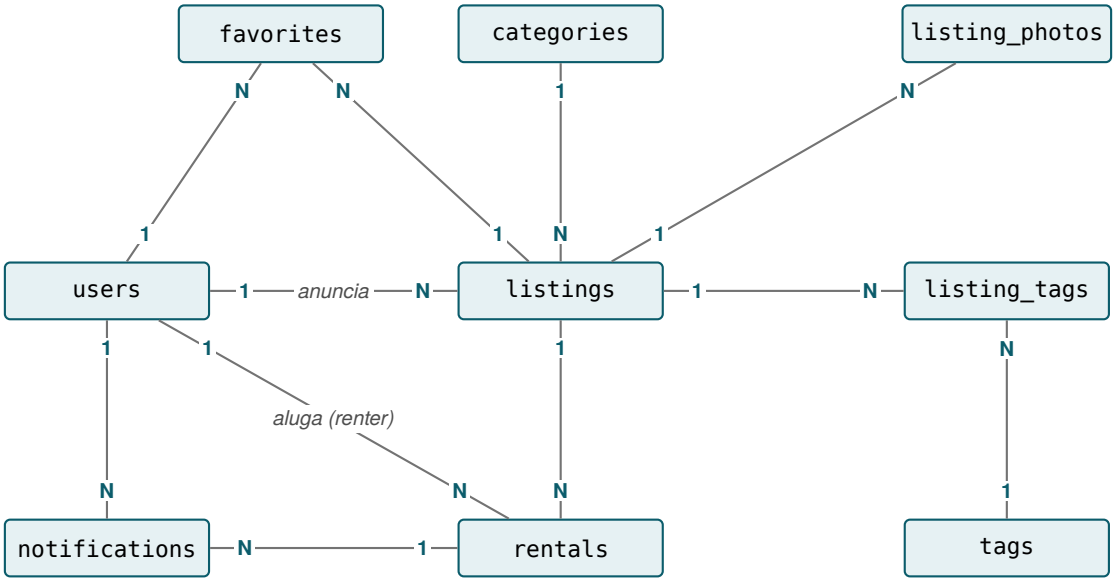
Backend: versões fixas em `uv.lock`, instaladas com `uv sync --frozen` (Dockerfile e CI). Frontend: `package-lock.json`, instalado com `npm ci`. Dependabot semanal para pip (uv), npm, Docker e GitHub Actions; atualização de série maior só com PR dedicado.

8.8 Tempo e datas

Uma única função `today_br()` (America/Sao_Paulo) define "hoje" em todo o backend, e o relógio é injetável nos testes. É proibido usar `date.today()` diretamente.

9 Modelo de dados

9.1 Diagrama



Extensões exigidas: `citext`, `pg_trgm`, `unaccent` e `btree_gist`, criadas na primeira migração (`CREATE EXTENSION IF NOT EXISTS`). A migração também cria a função `immutable_unaccent(text)`, um invólucro imutável de `unaccent` necessário para indexar a busca (RN-25).

9.2 Entidades

`users`

Coluna	Tipo	Restrições	Descrição
id	bigint identity	PK	
full_name	varchar(120)	NOT NULL	RN-06; privado
username	citext	UNIQUE, NOT NULL, `CHECK` de formato e tamanho (3 a 30)	RN-01, RN-05; o `CHECK` é dispensado após a anonimização
email	citext	UNIQUE, NOT NULL, até 254	RN-01, RN-02
cpf	varchar(11)	UNIQUE, NOT NULL, 11 dígitos	RN-03; dígitos verificadores validados na API; após a anonimização recebe valor inerte `x` + id com 10 dígitos
phone	varchar(11)	NOT NULL, 11 dígitos	RN-07; pode repetir
password_hash	varchar	NOT NULL	Argon2id (RN-10)
cep, street, number, complement, neighborhood, city, state	varchar (8, 100, 10, 60, 100, 100, 2)	NOT NULL (exceto `complement`)	RN-08
token_version	int	NOT NULL, padrão 0	Incrementa ao trocar a senha ou excluir a conta; invalida tokens antigos
created_at	timestamptz	NOT NULL	Membro desde (F-10)
deleted_at	timestamptz	NULL	Conta excluída
purge_after	timestamptz	NULL	`deleted_at` + 1 ano (RN-21)
anonymized_at	timestamptz	NULL	Preenchido pela anonimização

`categories` — `id`, `name` (UNIQUE). As 8 linhas fixas (RN-58) são inseridas por migração Alembic.

`listings` (anúncios)

Coluna	Tipo	Restrições	Descrição
id	bigint identity	PK	
owner_id	bigint	FK users, NOT NULL	
category_id	int	FK categories, NOT NULL	RN-58
title	varchar(80)	NOT NULL, tamanho ≥ 1	RN-53
description	varchar(1000)	NOT NULL	RN-53
daily_price	numeric(7,2)	NOT NULL, `CHECK` entre 1.00 e 5000.00	RN-32
max_days	int	NOT NULL, `CHECK` ≥ 1	RN-53
available_from, available_until	date	NOT NULL, `CHECK` from ≤ until	Janela de disponibilidade
—	—	`CHECK max_days` ≤ `available_until − available_from + 1`	RN-55
pickup_cep, pickup_street, pickup_number, pickup_complement, pickup_neighborhood, pickup_city, pickup_state	varchar	Cópia do cadastro por padrão	RN-59, RN-35
status	varchar	`CHECK` IN (`active`, `paused`, `deleted`)	A exclusão é lógica, para preservar o histórico
created_at, updated_at	timestamptz	NOT NULL	Ordenação e tempo desde a publicação

Regra que fica na API por depender de "hoje": `available_until` ≥ hoje na criação e na edição (RN-54).

`listing_photos` — `id`, `listing_id` (FK), `position` (1 a 5, `CHECK`), `filename`; UNIQUE (`listing_id`, `position`). A posição 1 é a capa (RN-56). O mínimo de 1 e o máximo de 5 são garantidos pela API.

`tags` — `id`, `name` (até 30), `normalized_name` UNIQUE (minúsculas e sem acento, RN-57). **`listing_tags`** — PK (`listing_id`, `tag_id`); o máximo de 5 por anúncio é garantido pela API.

`rentals` (propostas e aluguéis)

Coluna	Tipo	Restrições	Descrição
id	bigint identity	PK	
listing_id	bigint	FK, NOT NULL	
renter_id	bigint	FK users, NOT NULL	
start_date, end_date	date	NOT NULL, `CHECK` start ≤ end	Inclusivos (RN-36)
status	varchar	`CHECK` IN (`pending`, `accepted`, `rejected`, `expired`, `canceled`, `completed`)	Estado armazenado; a situação exibida é calculada (RN-65)
daily_price_snapshot	numeric(7,2)	NOT NULL	Fixo desde o envio (RN-43)
estimated_total	numeric(10,2)	NOT NULL	Preço × duração; recalculado na devolução antecipada (RN-50)
listing_snapshot	jsonb	NOT NULL	Cópia do anúncio no envio: título, descrição, preço, máximo de dias, categoria, tags, local de retirada e nomes dos arquivos de foto (RN-61)
created_at	timestamptz	NOT NULL	

Coluna	Tipo	Restrições	Descrição
expires_at	timestamptz	NOT NULL	`created_at` + 48 h (RN-47)
responded_at, canceled_at	timestamptz	NULL	
return_requested_at	timestamptz	NULL	Locatário avisou a devolução antecipada
returned_on	date	NULL	Data da devolução confirmada
original_end_date	date	NULL	Fim previsto original quando o período é encurtado

Restrição de exclusão (RN-42):

```
ALTER TABLE rentals ADD CONSTRAINT rentals_no_overlap
  EXCLUDE USING gist (
    listing_id WITH =,
    daterange(start_date, end_date, '[]') WITH &&
  ) WHERE (status IN ('accepted', 'completed'));
```

Os intervalos são **inclusivos** nas duas pontas: devolução em 14/10 só libera nova retirada em 15/10. Propostas `pending` **não** entram na restrição (RN-44).

`favorites` — PK (`user_id`, `listing_id`); o favorito de anúncio indisponível é mantido e fica oculto (RN-31).

`notifications` — `id`, `user_id`, `type`, `rental_id` (NULL), `payload` jsonb, `created_at` e `read_at` (NULL = não lida, RN-68). Índice parcial UNIQUE (`user_id`, `type`, `rental_id`) para os tipos `rental_starting`, `return_due` e `rental_overdue`, que torna o `worker` idempotente. Tipos e payloads: seção 5.

`login_attempts` — `id`, `identifier_hash` (SHA-256 do identificador normalizado, para não guardar e-mail ou usuário em claro), `attempted_at` e `success`. Usada no bloqueio por tentativas (RN-14).

9.3 Regras garantidas pelo banco

Regra	Mecanismo
Unicidade de e-mail, usuário e CPF (RN-01)	`citext` + `UNIQUE`
Formato do usuário (RN-05)	`CHECK` com expressão regular
Preço, prazo e janela coerentes (RN-32, RN-55)	`CHECK`
Sem sobreposição de aluguéis aceitos (RN-42)	`EXCLUDE USING gist`
Categoria obrigatória e fixa (RN-58)	FK + linhas inseridas por migração
Busca por qualquer parte do nome (RN-25)	Índice GIN `pg_trgm` sobre `immutable_unaccent(lower(title))`
Feed e filtros	Índices em `listings(status, available_until)`, `listings(created_at, id)`, `listings(daily_price, id)`, `rentals(listing_id, status)` e `rentals(renter_id, status)`

9.4 Evolução do schema

O schema é criado e evoluído pelo **Alembic**; a primeira migração cria extensões, função auxiliar, tabelas, índices e as categorias fixas. No início, o serviço `backend` executa `alembic upgrade head` antes de aceitar tráfego. As migrações são revisadas em PR, devem ser compatíveis com a versão anterior da API (rollback, seção 17) e são testadas no CI aplicando-as sobre um banco vazio.

9.5 Retenção e exclusão de conta

- **Exclusão:** `deleted_at` = agora, `purge_after` = `deleted_at` + 1 ano, `token_version` incrementado, anúncios do usuário passam a `paused` e o feed filtra donos excluídos (RN-20).
- **Reativação:** o login dentro do prazo limpa `deleted_at` e `purge_after`; os anúncios continuam `paused` até o dono reativá-los (RN-22).
- **Anonimização:** após `purge_after`, o `worker` substitui os dados pessoais por valores inertes (usuário `deleted-<id>`, e-mail `anon-<id>@invalid`, nome `Usuário excluído`, CPF `x` + id com 10 dígitos, telefone `00000000000`, endereço em branco e hash inutilizável) e preenche `anonymized_at`. A linha não é apagada, para não quebrar o histórico da outra parte (RN-21).
- Enquanto a conta está excluída, e-mail, CPF e usuário continuam em uso, o que impede conta duplicada.

10 Contrato da API

10.1 Convenções

Item	Regra
Base URL	`/api` (mesma origem do site)
Formato	JSON (`application/json`); criação e edição de anúncio em `multipart/form-data`
Nomes	`snake_case`
Datas	`YYYY-MM-DD` para períodos; instantes em ISO 8601, UTC
Valores	Decimal em texto com 2 casas, por exemplo `"12.50"`, em reais
Autenticação	Cookie `vv_session` (`HttpOnly; SameSite=Lax; Path=/`), seção 12
Paginação	`limit` (padrão 20, máximo 50) e `cursor` opaco; resposta `{"items": [...], "next_cursor": "..." ou null, "total": N}` (RN-30, F-24)
Erro	`{"error": {"code": "RENTAL_OVERLAP", "message": "...", "fields": {"campo": "mensagem"}}}`

10.2 Resumo dos endpoints

Método	Rota	Acesso	Finalidade
POST	`/api/auth/register`	público	Cadastro (F-01)
POST	`/api/auth/login`	público	Login; reativa conta excluída (F-05, F-14)
POST	`/api/auth/logout`	logado	Sair (F-08)
GET	`/api/auth/me`	logado	Usuário logado, com os dados próprios completos
GET	`/api/users/{username}`	logado	Perfil público (F-10)
PATCH	`/api/me`	logado	Editar dados (F-11)

Método	Rota	Acesso	Finalidade
PUT	`/api/me/password`	logado	Trocar senha (F-12)
DELETE	`/api/me`	logado	Excluir conta, com a senha (F-13)
GET	`/api/categories`	logado	Categorias fixas (F-25)
GET	`/api/tags`	logado	Tags populares ou busca por `?q=` (F-17)
GET	`/api/listings/price-ranges`	logado	Atalhos de preço (F-19)
GET	`/api/listings`	logado	Feed, busca e filtros (F-15 a F-26)
GET	`/api/listings/{id}`	logado	Página do item (F-28, F-31)
GET	`/api/listings/{id}/booked-periods`	logado	Períodos reservados (F-29)
POST	`/api/listings`	logado	Criar anúncio (F-38)
PATCH	`/api/listings/{id}`	dono	Editar anúncio, incluindo fotos (F-41)
POST	`/api/listings/{id}/pause` · `/resume`	dono	Pausar e reativar
DELETE	`/api/listings/{id}`	dono	Excluir anúncio
GET	`/api/me/listings`	logado	Meus anúncios, ativos e histórico (F-42, F-43)
PUT · DELETE	`/api/listings/{id}/favorite`	logado	Favoritar e desfavoritar (F-26)
POST	`/api/listings/{id}/rentals`	logado	Propor aluguel (F-34)
GET	`/api/me/rentals?role=renter|owner&scope=active|history`	logado	Itens que estou alugando e propostas recebidas (F-42 a F-45)
POST	`/api/rentals/{id}/accept` · `/reject`	dono	Aceitar e recusar (F-39)
POST	`/api/rentals/{id}/cancel`	locatário	Cancelar proposta ou aluguel (F-36)
POST	`/api/rentals/{id}/return-request`	locatário	Avisar devolução antecipada (F-37)
POST	`/api/rentals/{id}/complete`	dono	Marcar devolvido (F-40)
GET	`/api/notifications` · `/unread-count`	logado	Caixa e indicador (F-46, F-47)
POST	`/api/notifications/{id}/read` · `/read-all`	logado	Marcar como lidas (F-52)
GET	`/api/health`	público	Saúde (API e banco)
GET	`/uploads/{filename}`	—	Imagem, servida pelo Nginx

Não existe endpoint para o **dono** cancelar um aluguel aceito (RN-51).

10.3 Detalhes por endpoint (validações na ordem aplicada)

`POST /api/auth/register` — corpo: `full_name`, `username`, `email`, `password`, `cpf`, `phone`, `cep`, `street`, `number`, `complement?`, `neighborhood`, `city`, `state`.

1. Campos obrigatórios e tamanhos (RN-06, RN-08) → 422
2. Formato de e-mail (RN-02), usuário (RN-05), senha (RN-04), telefone (RN-07) e CEP (RN-08) → 422
3. CPF válido (RN-03) → 422
4. Unicidade de e-mail, CPF e usuário → 409 `FIELD_IN_USE`, com `fields` indicando qual campo (RN-09)
5. Grava o hash e responde **201** sem dados sensíveis; o front leva ao login (F-03).

`POST /api/auth/login` — corpo: `identifier` (e-mail ou usuário) e `password`.

1. Identificador bloqueado por tentativas (RN-14) → 429 `LOGIN_LOCKED`, com `Retry-After`
2. Busca ignorando maiúsculas (RN-11)
3. Credenciais inválidas **ou** conta inexistente → 401 `INVALID_CREDENTIALS`, com mensagem única (RN-12); o tempo de resposta é o mesmo, comparando com um hash fictício

4. Conta excluída dentro de 1 ano → reativa (RN-22)
5. Define o cookie `vv_session` com validade de 15 dias (RN-13) e responde **200**.

`PUT /api/me/password` — corpo: `current_password`, `new_password`. Senha atual incorreta → 401; nova senha inválida → 422. Ao concluir, incrementa `token_version` e emite novo cookie para a sessão atual.

`DELETE /api/me` — corpo: `password`.

1. Senha atual correta (RN-16) → 401
2. Sem aluguel `accepted` em curso ou futuro, como locatário ou dono (RN-18) → 409 `ACCOUNT_HAS_ACTIVE_RENTAL`
3. Cancela propostas pendentes, pausa anúncios e marca `deleted_at` (RN-19, RN-20, RN-21)
4. Limpa o cookie e responde **204**.

`GET /api/listings` — parâmetros: `q`, `categories[]`, `tags[]`, `price_min`, `price_max`, `availability=all|available_now|rented_now`, `needed_from`, `needed_until`, `favorites=true`, `sort=newest|oldest|price_asc|price_desc`, `limit`, `cursor`.

- Base (RN-24): anúncios `active`, `available_until` ≥ hoje e dono não excluído. Anúncios do próprio usuário aparecem com `is_owner: true`, e o card mostra "Seu anúncio".
- `q` **restringe sempre**, com `ILIKE` sobre `immutable_unaccent(lower(title))`, escapando `%`, `_` e `\` (RN-25).
- Filtros de **tipos diferentes** combinam com **E**; dentro do mesmo tipo vale **OU** (RN-26).
- `availability` conforme RN-28; o período só vale com `needed_from` e `needed_until` (RN-29).
- Ordenação com desempate por `id` e **cursor por chave** (não por página), para não repetir nem pular itens (RN-30).
- Cada item traz: foto de capa, situação (disponível ou alugado), categoria, título, `daily_price`, bairro e cidade, `created_at`, `is_owner` e `is_favorite` (F-23). Nunca devolve e-mail, CPF, telefone, nome completo nem endereço completo (RN-15).

`GET /api/listings/price-ranges` — resposta `{"ranges": [{"label": "Mais baratos", "min": "10.00", "max": "25.00"}, ...]}` com 3 faixas por tercis dos anúncios ativos (RN-27); lista vazia com menos de 3 anúncios ativos.

`GET /api/listings/{id}` — 404 se o anúncio não existe. Para anúncio `paused` ou `deleted` ou dono excluído, responde **200** com `unavailable: true` (a tela mostra "anúncio indisponível", RN-34), exceto para o dono, que o vê normalmente (F-31). Endereço: rua, bairro e cidade para todos; **número e complemento** só para o locatário com aluguel `accepted` no item (RN-35, RN-17), lidos da cópia `listing_snapshot` do aluguel. Para o dono, `is_owner: true` (RN-33).

`POST /api/listings` (multipart) — campos: `title`, `description`, `daily_price`, `max_days`, `available_from`, `available_until`, `category_id`, `tags[]`, `pickup_*?` e `photos[]`.

1. Obrigatórios e tamanhos (RN-53) → 422
2. `daily_price` entre 1,00 e 5000,00 (RN-32)
3. `available_from` ≤ `available_until`
4. Fim da janela não está no passado (RN-54)
5. `max_days` ≤ tamanho da janela; senão **recusa** (RN-55)
6. Categoria existente (RN-58)
7. Até 5 tags, normalizadas (RN-57)
8. Fotos: de 1 a 5, até 5 MB, tipo real validado (RN-56, seção 13)
9. Local de retirada padrão = cadastro (RN-59) · **201**.

`PATCH /api/listings/{id}` (multipart) — todos os campos da criação são opcionais; só os

enviados mudam. Para fotos: `keep_photo_ids[]` (ids a manter, na ordem desejada) e `photos[]` (novas, adicionadas ao final). A lista final precisa ter de 1 a 5 fotos, e a primeira é a capa. `tags[]` e `pickup_*`, quando enviados, substituem os valores atuais.

1. Só o dono (RN-60) → 403
2. Mesmas validações de campo da criação, aplicadas aos valores finais
3. Encurtar a janela de modo que um aluguel `accepted` em curso ou futuro deixe de caber nela → 409 `LISTING_HAS_ACTIVE_RENTAL` (RN-62)
4. Grava o anúncio. A edição **não altera** propostas e aluguéis existentes, que usam a própria `listing_snapshot` (RN-61); vale para novas propostas.
5. Fotos removidas só são apagadas do disco quando nenhuma `listing_snapshot` de aluguel `pending` ou `accepted` as referencia.

`POST /api/listings/{id}/pause` · `/resume` · `DELETE /api/listings/{id}` — só o dono (403). Pausar e excluir com aluguel `accepted` em curso ou futuro → 409 `LISTING_HAS_ACTIVE_RENTAL`; ao pausar ou excluir, as propostas pendentes são recusadas (`reason: listing_unavailable`) e os locatários avisados (RN-62). Reativar exige `available_until` ≥ hoje (senão 409).

`POST /api/listings/{id}/rentals` — corpo: `start_date`, `end_date`.

1. Anúncio existe, está `active` e o dono não está excluído → 404 ou 409 `LISTING_UNAVAILABLE`
2. Não é o próprio item (RN-40) → 403
3. Datas válidas e início ≤ fim
4. Início ≥ hoje (RN-39)
5. Período dentro da janela (RN-38)
6. Duração entre 1 e `max_days` (RN-36, RN-37)
7. Sem aluguel `accepted` ou `completed` sobreposto (RN-42) → 409 `RENTAL_OVERLAP`
8. Sem outra proposta `pending` do mesmo locatário, no mesmo item, com período sobreposto (RN-45) → 409 `DUPLICATE_PROPOSAL`
9. Grava `pending` com `daily_price_snapshot`, `estimated_total`, `listing_snapshot` e `expires_at` = agora + 48 h (RN-41, RN-43, RN-47)
10. Avisa o dono (`proposal_received`) · **201**.

Propostas pendentes de outros locatários não bloqueiam (RN-44). Locatário com aluguel atrasado pode propor normalmente (RN-52).

`POST /api/rentals/{id}/accept`

1. Só o dono (RN-63) → 403
2. `status = pending` e não vencida; se vencida, marca `expired` → 409 `PROPOSAL_EXPIRED`
3. `start_date` ≥ hoje; senão marca `expired` → 409 `PROPOSAL_EXPIRED` (RN-48)
4. **Na mesma transação:** muda para `accepted`, recusa automaticamente as pendentes sobrepostas (`reason: overlap`) e avisa os locatários (RN-46)
5. Se a restrição de exclusão disparar (aceite simultâneo) → 409 `RENTAL_OVERLAP`. A partir do aceite, telefone e endereço completo ficam visíveis à outra parte (RN-17, RN-35).

`POST /api/rentals/{id}/reject` — só o dono, sem motivo (RN-63); `pending` → `rejected` (`reason: owner`); avisa o locatário (F-49).

`POST /api/rentals/{id}/cancel` — só o locatário (o dono não cancela aluguel aceito, RN-51). `pending`: a qualquer momento; `accepted`: apenas se `start_date` > hoje (RN-49), senão 409 `CANCEL_NOT_ALLOWED`. Avisa o dono quando o aluguel era `accepted`.

`POST /api/rentals/{id}/return-request` — só o locatário; exige `accepted` e `start_date` ≤ hoje; grava `return_requested_at` e avisa o dono (RN-50).

`POST /api/rentals/{id}/complete` — só o dono (RN-64); exige `accepted` e `start_date` ≤ hoje. Se houve `return_requested_at` antes do fim, encurta o período e recalcula o valor (seção 4.1). Avisa o locatário (`rental_completed`). O atraso não gera multa.

`GET /api/me/rentals` — `role=renter` lista "Itens que estou alugando"; `role=owner` lista as propostas e os aluguéis dos meus anúncios; `scope=active|history` (RN-66). Cada linha traz a **situação calculada** (RN-65). O telefone da outra parte só vem com `accepted` (RN-17); contas excluídas aparecem como "usuário excluído" (RN-19).

10.4 Códigos HTTP

Código	Uso
200 · 201 · 204	Sucesso · criado · sem corpo
400	Requisição malformada (JSON inválido, cursor inválido)
401	Sem sessão, sessão expirada ou credencial incorreta
403	Autenticado sem permissão (não é o dono, próprio item)
404	Recurso inexistente
409	Conflito de regra (já em uso, sobreposição, estado inválido, conta com aluguel ativo)
413	Corpo ou arquivo acima do limite
415	Tipo de arquivo não aceito
422	Validação de campo (com `fields`)
429	Muitas tentativas de login (`Retry-After`)
500	Erro inesperado, sem detalhes ao cliente

10.5 Catálogo de códigos de erro

Código (`error.code`)	HTTP	Quando ocorre
`VALIDATION_ERROR`	422	Campo ausente, formato ou tamanho inválido (detalhe em `fields`)
`FIELD_IN_USE`	409	E-mail, CPF ou usuário já cadastrado
`INVALID_CREDENTIALS`	401	Login com credenciais que não conferem ou senha atual incorreta
`LOGIN_LOCKED`	429	Identificador bloqueado por tentativas
`UNAUTHENTICATED`	401	Sem sessão ou sessão expirada
`FORBIDDEN`	403	Ação restrita ao dono, ao locatário ou proibida no próprio item
`NOT_FOUND`	404	Recurso inexistente
`ACCOUNT_HAS_ACTIVE_RENTAL`	409	Exclusão de conta com aluguel aceito em curso ou futuro
`LISTING_UNAVAILABLE`	409	Proposta para anúncio pausado, excluído ou de conta excluída
`LISTING_HAS_ACTIVE_RENTAL`	409	Pausar, excluir ou encurtar janela com aluguel aceito em curso ou futuro
`RENTAL_OVERLAP`	409	Período sobreposto a aluguel aceito
`DUPLICATE_PROPOSAL`	409	Proposta pendente sobreposta do mesmo locatário
`PROPOSAL_EXPIRED`	409	Aceite de proposta vencida ou com início no passado
`INVALID_STATE`	409	Ação incompatível com o estado atual do aluguel
`CANCEL_NOT_ALLOWED`	409	Cancelamento de aluguel aceito a partir do dia do início

Código (`error.code`)	HTTP	Quando ocorre
`INVALID_CURSOR`	400	Cursor de paginação inválido
`FILE_TOO_LARGE`	413	Foto acima de 5 MB
`UNSUPPORTED_MEDIA`	415	Arquivo que não é JPEG, PNG ou WebP
`INTERNAL_ERROR`	500	Erro inesperado

10.6 Fonte do contrato

O OpenAPI gerado pelo FastAPI é a fonte do **formato** dos contratos; este documento descreve **regras e ordem de validação**. O comando `make openapi` exporta `docs/openapi.json`, e o CI falha se o arquivo versionado estiver desatualizado.

11 Frontend

Não há protótipo de telas. O design visual é livre, desde que respeite as rotas, os componentes, os comportamentos e as regras descritos neste documento.

11.1 Mapa de rotas e acesso

Rota	Tela	Acesso
`/login`	Login (F-05)	Pública; logado é redirecionado a `/`
`/cadastro`	Cadastro (F-01)	Pública; logado é redirecionado a `/`
`/`	Feed e busca (F-15 a F-27)	Privada
`/itens/:id`	Página do item (F-28 a F-31)	Privada
`/anunciar`	Criar anúncio (F-38)	Privada
`/anuncios/:id/editar`	Editar anúncio (F-41)	Privada, só dono
`/alugueis`	Meus anúncios · Itens que estou alugando (F-42 a F-45)	Privada
`/usuarios/:username`	Perfil público (F-10)	Privada
`/configuracoes`	Editar dados, trocar senha, excluir conta (F-11 a F-13)	Privada
`/notificacoes`	Caixa de notificações (F-47)	Privada
`*`	Página não encontrada	Pública

Rota privada sem sessão leva a `/login`, guardando a rota de destino. Uma resposta **401** da API em qualquer chamada limpa a sessão e redireciona ao login (RN-13).

11.2 Estado e acesso à API

- **Sessão:** contexto `AuthContext` com o usuário vindo de `GET /api/auth/me`; o cookie é `HttpOnly`, então o JavaScript não lê o token.
- **Serviço de API:** `services/api.js` é o **único ponto** que chama o backend (`fetch` com `credentials: 'same-origin'`, tratamento do formato de erro e redirecionamento no 401). Páginas e componentes nunca chamam `fetch` diretamente.
- **Dados do servidor:** TanStack Query, com chaves por recurso (`['listings', filtros]`,

`['rentals', role, scope]`, `['notifications']`). Ações (propor, aceitar, favoritar) invalidam as chaves afetadas.

- **Rolagem infinita** (RN-30): `useInfiniteQuery` com `next_cursor` e `IntersectionObserver`; o contador (F-24) vem do campo `total`.
- **Filtros na URL:** os filtros do feed ficam em query params, para que recarregar ou compartilhar o link mantenha a busca.
- **Notificações:** o frontend monta o texto de cada aviso a partir de `type` e `payload` (seção 5) e navega ao destino indicado (RN-67).

11.3 Componentes reutilizáveis

`ListingCard` (foto, situação, categoria, nome, valor por dia, bairro, tempo, coração e selo Seu anúncio), `FilterSidebar` (categorias, tags, preço, disponibilidade e período), `DateRangePicker` (início, fim e resumo "N dia(s) · R$ X", F-33), `StatusBadge` (situação calculada), `PhotoUploader` (1 a 5 fotos, capa e reordenação), `ListingForm` (criação e edição), `NotificationBell`, `ConfirmDialog`, `FormField` (rótulo e erro por campo) e `EmptyState` (feed sem resultados, com limpar filtros e atalho para anunciar, F-27).

11.4 Build e configuração

- Caminhos **relativos** (`/api`, `/uploads`); nenhuma URL de servidor no código.
- Vite em desenvolvimento: `server.proxy` encaminha `/api` e `/uploads` ao backend; `base: '/'`.
- Produção: `npm ci && npm run build`; o Nginx serve `dist/` com fallback para `index.html` (seção 14).
- O resumo de valor e as validações de campo (F-04, F-33) são auxílio visual; a decisão é sempre da API.

11.5 Acessibilidade e responsividade

Design mobile-first (feed e cartões em grade responsiva); contraste mínimo WCAG AA; campos com rótulo e mensagem de erro associada; navegação por teclado nos menus e diálogos; textos em português; datas em `dd/mm/aaaa` e valores em `R$ 0,00`.

12 Autenticação e segurança

12.1 Autenticação e sessão

Tema	Definição
Hash da senha	Argon2id com parâmetros padrão da biblioteca; aceita senhas de até 128 caracteres (RN-04)
Login	`identifier` (e-mail ou usuário, sem diferenciar caixa) e senha; erro único e genérico (RN-11, RN-12)
Token	JWT assinado (HS256, segredo `JWT_SECRET`) com as claims `sub` (id do usuário), `tv` (`token_version`), `iat` e `exp` (15 dias corridos, RN-13)
Cookie	`vv_session`, com `HttpOnly`, `SameSite=Lax`, `Path=/` e `Max-Age` de 15 dias; `Secure` conforme `COOKIE_SECURE`
Invalidação	Trocar a senha ou excluir a conta incrementa `token_version`, invalidando os tokens anteriores (a API compara `tv` com o valor do banco)
Bloqueio de tentativas	5 falhas em 15 minutos para o mesmo identificador bloqueiam o login por 15 minutos (RN-14); valores em variáveis de ambiente

Tema	Definição
Validação de entrada	Pydantic em toda entrada; consultas parametrizadas, nunca SQL concatenado; escape de `LIKE` (RN-25)
CORS	Não necessário (mesma origem); em desenvolvimento o proxy do Vite evita CORS; nunca `Access-Control-Allow-Origin: *`
CSRF	`SameSite=Lax` e rotas que alteram dados só aceitam `POST`, `PUT`, `PATCH` e `DELETE` com `Content-Type` JSON ou multipart

12.2 OWASP Top 10 aplicado ao MVP

Risco	Tratamento
Controle de acesso quebrado	Toda rota verifica dono ou participante no serviço; testes de 403 por rota
Falhas criptográficas	Argon2id; `JWT_SECRET` longo e fora do Git
Injeção	ORM e parâmetros; escape de `LIKE`; sem SQL concatenado
Design inseguro	Regras críticas no banco (`EXCLUDE`, `CHECK`)
Configuração incorreta	Contêiner sem root, banco só na rede interna, `/docs` desabilitado fora de dev
Componentes vulneráveis	Lockfiles, Dependabot e varredura no CI
Falhas de autenticação	Bloqueio por tentativas, erro genérico, expiração e invalidação por `token_version`
Integridade de software e dados	Imagens de base com tag fixa; CI obrigatório antes do merge
Logs e monitoramento	Logs estruturados sem dados pessoais; eventos de bloqueio e 403
SSRF	Sem chamadas a URLs informadas pelo usuário

12.3 Dados pessoais (LGPD)

CPF, telefone e endereço só saem da API para o próprio usuário e, no caso de telefone e endereço completo, para a outra parte de um aluguel `accepted`. Todos os campos públicos passam por **schemas de saída separados** (`PublicUser`, `PublicListing`); o modelo do banco nunca é reaproveitado como resposta. Logs não contêm dados pessoais. Coleta mínima: o CPF serve para evitar contas duplicadas e o telefone para combinar retirada e devolução.

Dado	Requisito	Quem pode ver
Senha	Nunca visível nem recuperável (RN-10)	Ninguém
E-mail	Protegido; serve para entrar na conta	Só o próprio usuário
CPF	Protegido; evita contas duplicadas	Só o próprio usuário
Telefone	Protegido; serve para combinar retirada e devolução	O próprio usuário e a outra parte de aluguel aceito (RN-17)
Endereço completo	Protegido; o local de retirada aparece sem número até o aceite (RN-35)	O próprio usuário; os demais veem rua, bairro e cidade, e o endereço completo após o aceite
Nome completo	Privado	Só o próprio usuário
Bairro e cidade	Públicos no perfil e no card do item	Todos
Nome de usuário	Público	Todos

Dados de contas excluídas são mantidos por 1 ano e depois anonimizados (RN-21).

12.4 Checklist de endurecimento para exposição fora da máquina local

O MVP roda apenas localmente. Antes de expor o sistema fora da máquina local, todos os itens abaixo devem estar cumpridos:

- ☐ HTTPS obrigatório no Nginx (certificado válido), redirecionamento de HTTP e `COOKIE_SECURE=true`
- ☐ Cabeçalhos `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy` e CSP básica
- ☐ Banco e backend **sem** portas publicadas no host; só o Nginx exposto
- ☐ `JWT_SECRET` e `POSTGRES_PASSWORD` trocados dos valores de exemplo; `.env` com permissão restrita
- ☐ `/docs` e `/openapi.json` desligados ou protegidos
- ☐ Limite de requisições no Nginx (`limit_req`) em `/api/auth/`
- ☐ Usuário não-root nos contêineres e imagens com tag fixa
- ☐ Backup do banco e das fotos testado (seção 17)
- ☐ Dados de exemplo (`seed_dev`) **não** carregados

13 Upload e armazenamento de imagens

Item	Definição
Formatos aceitos	JPEG, PNG e WebP
Tamanho e quantidade	Até **5 MB por foto** e de **1 a 5 fotos** por anúncio; a primeira é a capa (RN-56)
Validação	O tipo é conferido pelo **conteúdo** (assinatura do arquivo e decodificação com Pillow), não pelo `Content-Type` nem pela extensão
Reprocessamento	A imagem é regravada, removendo metadados EXIF, inclusive **localização GPS** (importante para a privacidade do endereço)
Nome do arquivo	UUID aleatório + extensão; nunca o nome enviado pelo usuário
Armazenamento	Volume nomeado `uploads`, montado em `/data/uploads` no backend e no worker e somente leitura no Nginx
Acesso	`GET /uploads/<arquivo>`, servido pelo Nginx com cache longo (nomes imutáveis)
Consistência em falha	Grava em arquivo temporário, insere no banco e move para o destino final; se o banco falhar, apaga os arquivos. Uma tarefa diária remove arquivos sem referência
Edição de anúncio	Remover, adicionar e reordenar fotos mantém entre 1 e 5 e reordena `position`. Uma foto removida só é apagada do disco quando nenhuma `listing_snapshot` de aluguel ativo a referencia (RN-61)
Limites coerentes	Nginx com `client_max_body_size 26m` (5 × 5 MB + campos); a API rejeita arquivo > 5 MB (413) e mais de 5 arquivos (422); o limite do Nginx é sempre maior que o da API

14 Ambiente e conteinerização

14.1 Variáveis de ambiente

Variável	Padrão	Usada por	Descrição
`POSTGRES_DB`	`vaievolta`	db, backend	Nome do banco
`POSTGRES_USER`	`vaievolta`	db, backend	Usuário do banco
`POSTGRES_PASSWORD`	obrigatória	db, backend	Senha do banco

Variável	Padrão	Usada por	Descrição
`DATABASE_URL`	montada pelo Compose	backend, worker	`postgresql+psycopg://usuario:senha@db:5432/banco`
`JWT_SECRET`	obrigatória	backend	Assina a sessão (mínimo de 32 caracteres)
`JWT_EXPIRES_DAYS`	`15`	backend	Duração da sessão (RN-13)
`COOKIE_SECURE`	`false`	backend	Só `true` com HTTPS
`LOGIN_MAX_ATTEMPTS`	`5`	backend	Falhas antes do bloqueio (RN-14)
`LOGIN_WINDOW_MINUTES`	`15`	backend	Janela de contagem das falhas
`LOGIN_LOCK_MINUTES`	`15`	backend	Duração do bloqueio
`PROPOSAL_TTL_HOURS`	`48`	backend, worker	Prazo da proposta (RN-47)
`ACCOUNT_RETENTION_DAYS`	`365`	backend, worker	Retenção de contas excluídas (RN-21)
`UPLOAD_DIR`	`/data/uploads`	backend, worker	Pasta das imagens
`MAX_UPLOAD_MB` · `MAX_PHOTOS`	`5` · `5`	backend	Limites de foto (RN-56)
`TZ`	`America/Sao_Paulo`	backend, worker	Fuso para "hoje" (RN-69)
`APP_ENV`	`dev`	backend	`dev`, `test` ou `prod`; só em `dev` liga `/docs` e permite o seed
`LOG_LEVEL`	`INFO`	backend, worker	Nível de log
`WEB_PORT`	`8080`	web	Porta publicada do Nginx

O arquivo `.env.example` contém **todas** estas variáveis com valores falsos e deve ser mantido em sincronia com esta tabela (o PR que adicionar uma variável atualiza os dois). O `.env` real é ignorado pelo Git. Segredos nunca são colocados em `ARG`, `ENV` ou camadas de imagem; entram só em tempo de execução, por variável de ambiente.

14.2 Serviços

Serviço	Imagem	Portas	Volumes	Healthcheck
`db`	`postgres:18-alpine`	5432 (só em dev)	`pgdata` → `/var/lib/postgresql`	`pg_isready -U $POSTGRES_USER`
`backend`	build de `backend/`	8000 (interna; publicada só em dev)	`uploads` → `/data/uploads`	`GET /api/health`
`worker`	mesma imagem do backend, comando do worker	—	`uploads`	Arquivo `/tmp/worker_heartbeat` com menos de 120 s
`web`	build de `frontend/` (Nginx)	`${WEB_PORT}:80`	`uploads` (somente leitura)	`wget -q --spider http://localhost/`

14.3 Dockerfiles

- **Backend (múltiplos estágios):** `python:3.14-slim` com tag fixa; o estágio de build instala o `uv` (versão fixada, via `pip`) e executa `uv sync --frozen --no-dev`; o estágio final copia só o necessário, roda como **usuário sem privilégios** e inicia `uvicorn`. Camadas ordenadas para cache (primeiro `pyproject.toml` e `uv.lock`, depois o código).

- **Frontend (múltiplos estágios):** `node:24-alpine` para `npm ci && npm run build`; o estágio final usa `nginx:1.30-alpine` com `dist/` e `nginx.conf`.
- **`.dockerignore`** em ambos: `.git`, `node_modules`, `__pycache__`, `.env`, `tests` e `docs`.
- Nenhum segredo em `ARG`, `ENV` ou camadas.

14.4 Nginx (`frontend/nginx.conf`)

```
server {
  listen 80;
  client_max_body_size 26m;                # 5 fotos x 5 MB + campos (seção 13)

  gzip on;
  gzip_types text/css application/javascript application/json image/svg+xml;

  root /usr/share/nginx/html;

  location /api/ {
    proxy_pass http://backend:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location /uploads/ {
    alias /data/uploads/;
    expires 30d;
    add_header Cache-Control "public, immutable";
    add_header X-Content-Type-Options nosniff;
  }

  location /assets/ {                      # arquivos com hash do Vite
    expires 1y;
    add_header Cache-Control "public, immutable";
  }

  location / {
    try_files $uri /index.html;            # fallback da SPA
    add_header Cache-Control "no-cache";
  }
}
```

14.5 Compose

- **Rede interna** única; só o `web` publica porta na demonstração.
- **Ordem de subida** por `healthcheck`: `db` saudável, depois `backend` (que aplica `alembic upgrade head` ao iniciar) e `worker`, e por fim `web` (`depends_on` com `condition: service_healthy`).
- **Volumes nomeados:** `pgdata` e `uploads`. **Reinício:** `restart: unless-stopped` na demonstração.
- `docker-compose.yml` — base, com os 4 serviços, volumes e healthchecks.
- `docker-compose.override.yml` — **desenvolvimento**, carregado automaticamente: publica 8000 e 5432, monta o código com hot reload e usa `APP_ENV=dev`.
- `docker-compose.prod.yml` — **demonstração**: sem portas de banco e API, `APP_ENV=prod`, `restart` e limites de recurso; as imagens são construídas localmente (`build`).

14.6 Comandos oficiais

Ação	Comando
Subir (desenvolvimento)	`make up` (`docker compose up -d --build`)

Ação	Comando
Subir (demonstração)	`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`
Parar, mantendo dados	`make down`
Resetar tudo (apaga banco **e** fotos)	`make reset`
Lint · testes	`make lint` · `make test`
Dados de exemplo	`make seed`
Backup · restauração	`make backup` · `make restore DIR=backups/<carimbo>`
Exportar contrato OpenAPI	`make openapi`
Ver logs	`docker compose logs -f backend` (ou `worker`, `web`, `db`)
Console SQL	`docker compose exec db psql -U $POSTGRES_USER $POSTGRES_DB`

`Makefile` (os recuos das receitas são feitos com tabulação; aqui as receitas aparecem na mesma linha):

```
up:             ; docker compose up -d --build
down:           ; docker compose down
reset:          ; docker compose down -v && docker compose up -d --build
lint-backend:   ; cd backend && uv run ruff check . && uv run ruff format --check .
lint-frontend:  ; cd frontend && npm run lint && npm run format:check
lint:           lint-backend lint-frontend
test-backend:   ; cd backend && uv run pytest --cov=app --cov-report=xml
test-frontend:  ; cd frontend && npm test -- --coverage
test:           test-backend test-frontend
seed:           ; docker compose exec backend python -m app.scripts.seed_dev
backup:         ; ./scripts/backup.sh
restore:        ; ./scripts/restore.sh $(DIR)
openapi:        ; cd backend && uv run python -m app.scripts.export_openapi > ../docs/openapi.json
```

- `make backup` grava, em `backups/<AAAAMMDD-HHMMSS>/`, o dump do banco (`pg_dump -Fc`, via `docker compose exec -T db`) e o `tar` da pasta `/data/uploads`; mantém os 5 backups mais recentes. `backups/` fica no `.gitignore`.
- `make restore DIR=...` restaura o dump com `pg_restore --clean --if-exists` e extrai o `tar` das fotos.
- `make seed` é idempotente: pode ser executado várias vezes sem duplicar dados, e só funciona com `APP_ENV=dev`.

14.7 Dados de exemplo (seed)

As categorias fixas são criadas pela migração inicial e existem em todos os ambientes. Os dados de exemplo são criados por `make seed` (script `app/scripts/seed_dev.py`, que gera os hashes de senha em tempo de execução): 5 usuários, cerca de 15 anúncios com fotos de exemplo e tags, e aluguéis em cada situação (aguardando, reservado, em andamento, atrasado e concluído). Os usuários e a senha de desenvolvimento estão no Apêndice 19. **Nunca** usar esses dados fora de desenvolvimento e demonstração.

14.8 Problemas comuns

Sintoma	Causa	Solução
Página em branco ao recarregar `/alugueis`	Nginx sem `try_files ... /index.html`	Conferir o bloco `location /`
413 ao enviar fotos	Limite do Nginx menor que o enviado	`client_max_body_size 26m`
Sem dados de exemplo	O seed não roda automaticamente	`make seed`

Sintoma	Causa	Solução
502 Bad Gateway em `/api`	Backend ainda subindo ou falhou	`docker compose logs backend` e conferir o healthcheck
Login funciona e logo cai	`COOKIE_SECURE=true` em HTTP	Usar `false` no ambiente local
Fotos somem após `reset`	O volume `uploads` também é apagado	Usar `make down` em vez de `make reset`
Datas "um dia errado"	`TZ` ausente	Definir `TZ=America/Sao_Paulo`

15 Testes e validação técnica

15.1 Níveis e ferramentas

Nível	Prefixo	Ferramenta	O que cobre
Unitário	`UT-`	pytest · Vitest	Validadores (CPF, senha, usuário, telefone, CEP), duração e valor, função de situação (RN-65), normalização de tags
Banco	`DB-`	pytest + PostgreSQL real	`UNIQUE`, `CHECK` e `EXCLUDE` (conflito de datas)
Integração da API	`API-`	pytest + httpx + PostgreSQL real	Rotas, ordem das validações, permissões, serialização de dados privados e `worker`
Ponta a ponta	`E2E-`	Playwright	Fluxos principais pelo navegador, com Compose
Manual guiado	`MAN-`	Roteiro	Checklist do ambiente, segurança e uploads reais

Formato do identificador: `<NÍVEL>-<ÁREA>-<nn>`. Áreas das regras: `REG` cadastro · `AUT` login · `USR` conta · `FEED` feed · `ITM` item · `RNT` aluguel · `ADS` anúncio · `LST` situação e histórico · `NTF` notificações · `GER` regras gerais. Áreas adicionais para testes que não nascem de uma regra: `JOB` worker · `SEC` segurança · `IMG` imagens. Os identificadores da seção 4 são **reservados**: cada caso de teste leva o ID no nome da função ou do arquivo (`test_API_RNT_04_...`).

15.2 Dados de teste e isolamento

- **Fixtures** (pytest): fábricas de usuário, anúncio e aluguel; nenhum teste depende da ordem de execução.
- **Isolamento:** banco de teste separado; cada teste roda em transação com rollback (ou `TRUNCATE` entre testes que usam commit, como os de concorrência).
- **Relógio congelado:** `freezegun` ou relógio injetável em `today_br()`, obrigatório para RN-39, RN-47, RN-49, RN-48 e RN-65.
- **Concorrência** (RN-42, RN-46): teste com duas sessões aceitando propostas sobrepostas ao mesmo tempo; exatamente uma deve vencer.
- **Worker:** testes chamam cada tarefa diretamente, com o relógio congelado, e verificam que reexecutar não duplica avisos.

15.3 Roteiros de ponta a ponta

1. **Anunciar e alugar:** A cadastra-se e faz login → anuncia item com 2 fotos → B cadastra-se e vê o item no feed → B propõe de 12/10 a 14/10 → A recebe aviso e aceita → B vê telefone de A e endereço completo → A marca devolvido → ambos veem o aluguel no histórico.
2. **Busca por período:** com os dados de exemplo, filtrar "preciso de … até …"; confirmar que só aparecem itens cuja janela cobre o período e sem aluguel sobreposto; combinar com busca por

nome e ordenação; rolar o feed sem repetir itens.

3. **Conflito:** duas propostas para o mesmo período → A aceita uma → a outra é recusada automaticamente e o locatário é avisado.
4. **Conta:** tentar excluir com aluguel ativo (bloqueado) → concluir → excluir → anúncios somem do feed → logar de novo reativa a conta.
5. **Edição:** A edita o preço de um anúncio com aluguel em andamento → o aluguel mantém o valor e os dados antigos; uma nova proposta usa o novo preço.
6. **Devolução antecipada:** B avisa a devolução → A confirma → o período restante fica livre e o valor é recalculado.

15.4 Checklist de verificação do ambiente

"O projeto roda pelas instruções do repositório" significa:

- [ ] sobe do zero com um comando, apenas com Docker instalado;
- [ ] a aplicação abre e recarregar uma rota interna não gera página em branco;
- [ ] o upload de 5 fotos no tamanho máximo funciona;
- [ ] dados e fotos persistem após `make down` e nova subida;
- [ ] `make reset` recria banco e fotos do zero;
- [ ] o `worker` está em execução e expira uma proposta de teste (reduzir `PROPOSAL_TTL_HOURS`);
- [ ] login com 5 senhas erradas bloqueia e retorna 429.

15.5 Cobertura e relatórios

O CI gera relatório de cobertura (backend com `pytest --cov`, frontend com `vitest --coverage`) como artefato do job. **Meta fixa:** 100% das regras de negócio da seção 4 com ao menos um teste, e cobertura de linhas do backend de pelo menos 70% nas camadas `services` e `routers`.

16 Integração contínua (CI)

O Kit do Desenvolvedor define **o que** o CI deve verificar em cada Pull Request; esta seção define **como** isso é implementado, com GitHub Actions. Há um único workflow, `ci.yml`, executado em Pull Requests e em pushes na `main`. Não há etapa de entrega: o projeto não publica imagens nem faz deploy automático.

Verificação exigida	Job	Bloqueia merge
Linter e formatação	`lint-backend`, `lint-frontend`	Sim
Testes automatizados	`test-backend` (com PostgreSQL), `test-frontend`	Sim
Build	`build-frontend`, `build-images` (constrói as imagens sem publicar)	Sim
Segredos expostos	`secrets-scan`	Sim
Contrato OpenAPI atualizado	Passo dentro de `test-backend`	Sim

Os **nomes dos jobs** acima são os *required status checks* da proteção da `main`; renomear um job exige atualizar a proteção.

16.1 Detalhes dos jobs

- Instalação com cache (`setup-python` e `setup-node`); o backend usa `uv sync --frozen` e o frontend `npm ci`.
- `test-backend`: PostgreSQL 18 como `services:` do job; aplica as migrações (Alembic) sobre banco vazio e executa `make test-backend`; depois confere `docs/openapi.json`.
- `build-frontend`: `npm ci && npm run build`. `build-images`: `docker build` do backend e do frontend, **sem** `push`.
- `secrets-scan`: executa o binário de código aberto do Gitleaks (versão fixada) sobre o histórico do PR. Usa-se o binário, e não a action oficial, porque a action exige licença paga em repositórios de organização.
- **E2E (Playwright):** job separado `e2e`, executado na `main` e em horário noturno, fora da lista de checks obrigatórios, para manter o tempo do PR baixo.

16.2 Esqueleto do workflow

```
# .github/workflows/ci.yml (esqueleto)
name: ci
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  lint-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.14" }
      - run: pip install "uv==0.9.*"
      - run: cd backend && uv sync --frozen
      - run: make lint-backend
  test-backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:18-alpine
        env: { POSTGRES_PASSWORD: test, POSTGRES_DB: test }
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres" --health-interval 5s
          --health-timeout 5s --health-retries 10
    env:
      DATABASE_URL: postgresql+psycopg://postgres:test@localhost:5432/test
      APP_ENV: test
      JWT_SECRET: ci-only-secret-ci-only-secret-ci-only
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.14" }
      - run: pip install "uv==0.9.*"
      - run: cd backend && uv sync --frozen
      - run: cd backend && uv run alembic upgrade head
      - run: make test-backend
      - run: make openapi && git diff --exit-code docs/openapi.json
  secrets-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
        with: { fetch-depth: 0 }
      - run: |
          curl -sSL
          ↪  https://github.com/gitleaks/gitleaks/releases/download/v8.28.0/gitleaks_8.28.0_linux_x64.tar.gz
          ↪  | tar -xz gitleaks
          ./gitleaks git --redact --verbose
  # lint-frontend, test-frontend, build-frontend e build-images seguem o mesmo padrão
```

16.3 Custo, permissões e proteções

- **Sem custo:** o GitHub Actions é gratuito em repositórios públicos e tem cota mensal gratuita em privados; o pipeline é dimensionado para caber nela. Nenhum job usa serviço pago.
- **Permissões mínimas:** `contents: read` em todo o workflow; sem segredos de repositório, já que o CI usa apenas valores de teste definidos no próprio arquivo.
- **Dependabot** (pip/uv, npm, Docker e GitHub Actions) é gratuito e fica ativo. *Secret scanning* e *push protection* nativos do GitHub são gratuitos em repositório público; em repositório privado, a proteção contra segredos é feita pelo `secrets-scan`.
- **Proteção da `main`:** PR obrigatório, *required status checks* acima e bloqueio de push direto, conforme o Kit do Desenvolvedor. Esses recursos são gratuitos em repositório público.
- **Meta de tempo:** pipeline de PR em até 10 minutos, com jobs em paralelo.

16.4 Segredos e rotação

Nome	Para que serve	Onde fica	Quem rotaciona
`POSTGRES_PASSWORD`	Senha do banco	`.env` local	Responsável técnico
`JWT_SECRET`	Assina a sessão	`.env` local	Responsável técnico
Demais variáveis	Configuração não sensível	`.env` local	Responsável técnico

Segredo	Procedimento técnico
`JWT_SECRET`	Gerar novo valor, atualizar o `.env` e reiniciar o `backend`. **Efeito:** todos os usuários são deslogados
`POSTGRES_PASSWORD`	`ALTER USER ... PASSWORD ...` no banco, atualizar o `.env` e reiniciar `backend` e `worker`

17 Ambientes e operação

O projeto roda **somente localmente, via Docker**: não há servidor de deploy nem ambiente de teste separado.

Ambiente	Compose	Quem acessa	Portas expostas	Dados
Local (desenvolvimento)	`docker-compose.yml` + override	Devs e QA	8080, 8000, 5432	`seed_dev`
Demonstração	`docker-compose.prod.yml`	Banca e PO, na máquina local	Só o Nginx (`WEB_PORT`)	`seed_dev`

17.1 Subir a demonstração

1. Fazer checkout da tag da release (`vX.Y.Z`) e criar o `.env` a partir do `.env.example`, com valores próprios.
2. Executar `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`.
3. O `backend` aplica `alembic upgrade head` ao iniciar; em seguida, `make seed` carrega os dados de exemplo.
4. Verificar `GET /api/health` (200), `docker compose ps` (todos saudáveis) e os logs do `worker`.

17.2 Rollback

Voltar ao checkout da tag anterior e subir novamente. Se a versão nova aplicou migração incompatível com a anterior, restaurar o backup feito antes da subida (`make restore`).

17.3 Backup e restauração

Dado	Como	Quando
Banco	`make backup` (`pg_dump -Fc` via `docker compose exec`)	Antes de cada atualização e antes da apresentação
Fotos	`make backup` (`tar` da pasta `/data/uploads`)	Junto com o banco
Teste de restauração	`make restore` em ambiente limpo e abrir a aplicação	**Antes da apresentação**

Backups ficam fora do repositório, porque contêm dados pessoais, e apenas os 5 mais recentes são mantidos.

17.4 Logs e monitoramento mínimo

- Logs: `docker compose logs -f <serviço>` (JSON, sem dados pessoais).
- Observar: `GET /api/health`, estado dos 4 serviços, espaço em disco (volumes `pgdata` e `uploads`), erros 5xx, bloqueios de login (429) e atividade do `worker`.

17.5 Reset de dados

`make reset` apaga **o banco e as fotos**. Antes de usá-lo na demonstração, executar `make backup`.

17.6 Moderação

Não há ferramenta de moderação no MVP. Um anúncio indevido é removido manualmente pelo responsável técnico, alterando `status` para `deleted` diretamente no banco.

18 Limitações conhecidas

Tema	Detalhe	Mitigação
Validação de imagem	Tipo checado pelo conteúdo, mas sem antivírus	Reprocessar com Pillow e limitar tamanho (seção 13)
Armazenamento local	Fotos em volume do host, sem redundância	Backup manual (seção 17)

Tema	Detalhe	Mitigação
Fuso horário	"Hoje" fixo em America/Sao_Paulo	Função única `today_br()`
Reservas simultâneas	Dois aceites ao mesmo tempo	`EXCLUDE` no banco; erro `23P01` tratado como 409
Tarefas em segundo plano	`worker` parado atrasa os avisos	Consultas tratam proposta vencida como expirada; healthcheck do worker
Busca simples	`ILIKE` com trigram, sem ordenação por relevância	Suficiente para o volume do MVP
Atraso sem consequência	Locatário atrasado pode alugar outros itens	Atraso aparece como situação (RN-52)
Dependência de contato direto	O dono não pode retomar o item durante a locação	Combinação por telefone após o aceite (RN-51)

19 Decisões técnicas adotadas

Resumo das decisões que valem para todo o projeto. Mudanças futuras nessas decisões devem ser alinhadas com o responsável técnico e refletidas neste documento.

\#	Decisão	Valor adotado
1	Idioma dos nomes	Identificadores em inglês; textos ao usuário em português
2	Ferramentas de lint, formatação e testes	Ruff · ESLint + Prettier · pytest · Vitest · Playwright
3	Gerenciamento de dependências	Backend com uv e `uv.lock` (`uv sync --frozen`); frontend com `package-lock.json` (`npm ci`)
4	Driver do PostgreSQL	psycopg 3 com SQLAlchemy 2.0
5	Evolução do schema e dados fixos	Alembic; extensões e categorias criadas pela migração inicial; `alembic upgrade head` no início do backend
6	Contrato da API	OpenAPI exportado para `docs/openapi.json` e conferido no CI
7	Entrega e registro de imagens	Sem registro de imagens e sem deploy; imagens construídas localmente; CI apenas valida o build
8	Serviços gratuitos	Somente ferramentas gratuitas e de código aberto; Gitleaks por binário, sem action paga
9	Varredura de segredos	Gitleaks no CI, mais secret scanning nativo quando o repositório for público
10	Cobertura de testes	100% das regras com teste; pelo menos 70% de linhas em `services` e `routers`
11	Backup	`make backup` manual antes de atualizar e da apresentação; teste de restauração antes da apresentação
12	Sessão	JWT HS256 no cookie `vv_session` (`HttpOnly`), 15 dias; claims `sub`, `tv`, `iat`, `exp`
13	Bloqueio de login	5 falhas em 15 minutos, por identificador, bloqueiam por 15 minutos
14	Retenção de conta excluída	1 ano; depois anonimização (`deleted-<id>`, `anon-<id>@invalid`), preservando o histórico da outra parte
15	Combinação de filtros do feed	E entre tipos de filtro; OU dentro do mesmo tipo
16	Edição de anúncio	Todos os campos editáveis; cada proposta guarda uma cópia do anúncio (`listing_snapshot`), então a edição vale só para novos locatários
17	Faixas de preço	3 faixas por tercis dos anúncios ativos
18	Tamanhos de campos	Nome 120 · usuário 3 a 30 · e-mail 254 · senha 8 a 128 · telefone 11 dígitos · CEP 8 dígitos · rua, bairro e cidade 100 · número 10 · complemento 60 · UF 2
19	Termos de uso e idade mínima	Fora do MVP; o cadastro não coleta aceite nem idade

\#	Decisão	Valor adotado
20	Retomada do item pelo dono	Fora do MVP; combinação por telefone após o aceite
21	Moderação	Remoção manual pelo responsável técnico, direto no banco
22	Moeda e fuso	Sempre BRL e America/Sao_Paulo
23	Histórico de aluguéis	Visível enquanto a conta existir, sem prazo de expiração
24	Healthcheck do worker	Arquivo de heartbeat atualizado a cada ciclo e lock consultivo do PostgreSQL contra instâncias duplicadas

Apêndice A — Usuários de exemplo

Dados fictícios carregados por `make seed`, para desenvolvimento e demonstração. Todos os usuários têm a mesma senha de desenvolvimento, que cumpre RN-04: **`Dev@12345`**. Os CPFs têm dígitos verificadores válidos, mas são fictícios; os e-mails usam o domínio reservado `.test`.

Usuário	Nome	E-mail	CPF	Telefone	Bairro e cidade
ana_silva	Ana Silva	ana.silva@exemplo.test	11144477735	53991110001	Centro, Rio Grande
bruno_costa	Bruno Costa	bruno.costa@exemplo.test	12345678909	53991110002	Cassino, Rio Grande
carla_souza	Carla Souza	carla.souza@exemplo.test	24681357928	53991110003	Cidade Nova, Rio Grande
diego_lima	Diego Lima	diego.lima@exemplo.test	13579246828	53991110004	Parque Marinha, Rio Grande
elisa_rocha	Elisa Rocha	elisa.rocha@exemplo.test	98765432100	53991110005	Centro, Rio Grande

Cenário mínimo do seed: Ana e Bruno anunciam a maior parte dos itens; Carla, Diego e Elisa alugam. Há aluguéis nas situações aguardando, reservado, em andamento, atrasado e concluído, além de uma conta excluída dentro do prazo de reativação para testar RN-22.

Histórico de revisões

Versão	Data	Descrição
1.0	10/10/2026	Versão inicial